H4: Beslissingen

Labo 1 · week 4

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Waarom beslissingen?** Van lineaire programma's naar programma's die kiezen
2. **Booleaanse expressies**: vergelijken met `<`, `>`, `==`, `!=`, …
3. `if`: code die enkel soms uitvoert
4. `if/else`: een plan B
5. **Nesting en** `else if`: meerdere gevallen afhandelen



Vandaag leren we beslissingen nemen. Dat is meer dan sommigen aan de toog van de frituur kunnen.

Waarom beslissingen?

Tot nu toe: lineair

Onze programma's liepen braaf **regel per regel** van boven naar onder. Zoals een algoritme om brood te kopen:

1. Neem geld uit spaarpot
2. Wandel naar de bakker om de hoek
3. Vraag om een brood
4. Krijg het brood
5. Betaal het geld aan de bakker
6. Keer huiswaarts
7. Smullen maar

😈 *Stap 7: het enige deel van het algoritme dat nooit faalt.*

Programma's die kiezen

Een beter algoritme reageert op de **omstandigheden**: is de bakker gesloten? Is er genoeg geld?

Het programma neemt beslissingen op basis van aanwezige informatie.

Voorbeelden in een bestelsysteem:

- reageren als de voorraad van een product op is
- korting geven op bepaalde dagen van de week
- spaarpunten laten inwisselen



Een programma zonder beslissingen is als een gps die altijd "rechtdoor" zegt.

Booleaanse expressies

Relationele operatoren

Een **booleaanse expressie** is een stukje code dat `true` of `false` oplevert.

C#-syntax	Betekenis
`>`	groter dan
`<`	kleiner dan
`==`	gelijk aan
`!=`	niet gelijk aan
`<=`	kleiner dan of gelijk aan
`>=`	groter dan of gelijk aan



Relationele operatoren: geen datingapp, wel vergelijkingen.

`=` of `==`?

```
1  int age = 18;                    // toekenning: age wordt 18
2  Console.WriteLine(age == 18);    // vraag: is age gelijk aan 18?
```

> ⚠️ **WAARSCHUWING**
>
> "Gelijk aan" schrijf je met **twee** symbolen, omdat één `=` al gebruikt wordt voor een toekenning.

😈 *Eén `=` geeft een bevel, twee `==` stellen een vraag. Drie `===` is JavaScript, en daar praten we hier niet over.*

Uittesten

```
1  Console.WriteLine(4 < 7);   // True
2  Console.WriteLine(4 < 2);   // False
3  Console.WriteLine(5 == 5);  // True
4  Console.WriteLine(5 != 5);  // False
```

Het resultaat kan je ook bewaren in een variabele van het type `bool`:

```
1  int age = 17;
2  bool isAdult = age >= 18;
3  Console.WriteLine(isAdult);  // False
```

C# schrijft `True` met een hoofdletter. Het is nu eenmaal zeker van zijn stuk.

Even testen 🤔

```
1  int number = 10;
```

Welke expressies leveren `true` op?

1. `number > 10` → ❌ `false`
2. `number >= 10` → ✅ `true`
3. `number != 3` → ✅ `true`
4. `number < 5` → ❌ `false`
5. `"C#" == "c#"` → ❌ `false`

😈 *`4 < 7` is `true`. Net als "maandag komt altijd te vroeg".*

Appelen met peren

De types die je vergelijkt, moeten **vergelijkbaar** zijn:

```
1  if ("4" > 3)   // ❌ compileert niet: string met int vergelijken
```

Zet eerst om, dan vergelijk je getallen met getallen:

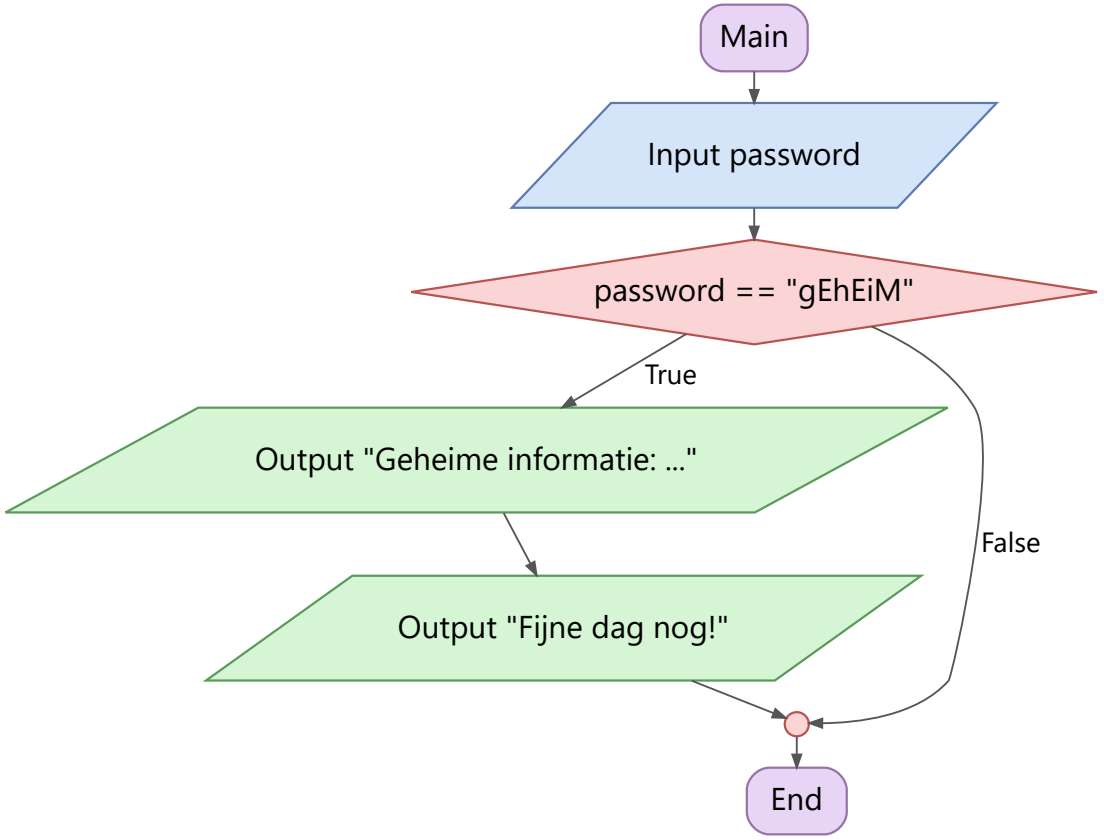
```
1  if (Convert.ToInt32("4") > 3)   // ✅
```

🐞 *De compiler vergelijkt geen appelen met peren. Hij is geen fruitboer.*

`if`

Het idee

Een `if` bevat code die **enkel** uitvoert als een booleaanse expressie `true` oplevert. Dat heet **conditionele code**.



Van schema naar C#

```
1  string password = Console.ReadLine();
2  if (password == "gEhEiM")
3  {
4      Console.WriteLine("Geheime informatie: ...");
5      Console.WriteLine("Fijne dag nog!");
6  }
```

- De **ruit** = de voorwaarde tussen de **ronde haakjes** na `if`
- De tak **True** = de code tussen de **accolades**
- De tak **False** = gewoon verder na de `}`

Een flowchart is een stripverhaal voor programmeurs. Alleen met meer ruiten en minder superhelden.

Met invoer van de gebruiker

```
1  Console.WriteLine("Hoe oud ben je?");
2  int age = Convert.ToInt32(Console.ReadLine());
3  if (age >= 18)
4  {
5      Console.WriteLine("Je mag stemmen.");
6  }
7  Console.WriteLine("Bedankt!");
```

```
Hoe oud ben je?
16
Bedankt!
```

In België is stemmen voor de meeste verkiezingen zelfs een plicht. Ook dat is een `if`.

Accolades vergeten

```
1  if (password == "gEhEiM")
2      Console.WriteLine("Geheime informatie: ...");
3      Console.WriteLine("Fijne dag nog!");
```

Zonder accolades hoort **enkel de eerste regel** bij de `if`. De tweede regel wordt **altijd** uitgevoerd, ook met een fout wachtwoord. Het inspringen verandert daar niets aan.

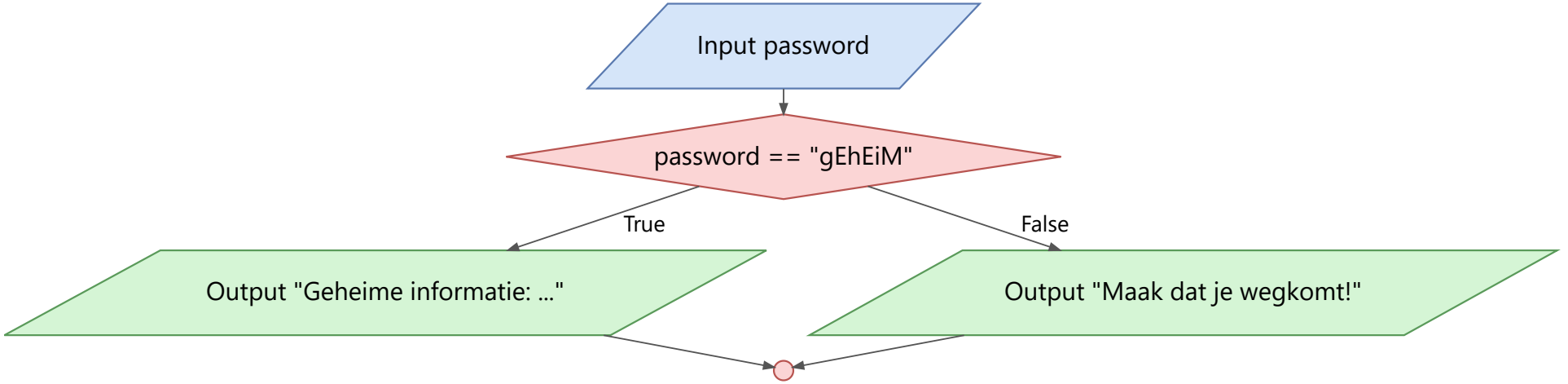
> **BELANGRIJK**
>
> Voor ons is het simpel: we schrijven `if` **altijd** met accolades.

Accolades vergeten is als je broek vergeten: technisch kan je vertrekken, maar het loopt slecht af.

if/else

Een plan B

Vaak wil je iets **anders** doen als niet aan de voorwaarde voldaan is.



`if/else`: *het enige moment waarop een computer een plan B heeft.*

`if/else` in C#

```
1 if (password == "gEhEiM")
2 {
3     Console.WriteLine("Geheime informatie: ...");
4 }
5 else
6 {
7     Console.WriteLine("Maak dat je wegkomt!");
8 }
```

- De tak **True** = het eerste blok accolades
- De tak **False** = het blok na `else`
- Er wordt **altijd precies één** van de twee uitgevoerd

Code voor beide gevallen

Wat je **altijd** wil doen, hoort niet in een vertakking. Zet het **na** de volledige `if/else`:

```
1 if (password == "gEhEiM")
2 {
3     Console.WriteLine("Geheime informatie: ...");
4 }
5 else
6 {
7     Console.WriteLine("Maak dat je wegkomt!");
8 }
9 Console.WriteLine("Fijne dag nog!");
```

😈 *"Fijne dag nog!" krijg je altijd. Zelfs als je net betrapt bent.*

Nesting en `else if`

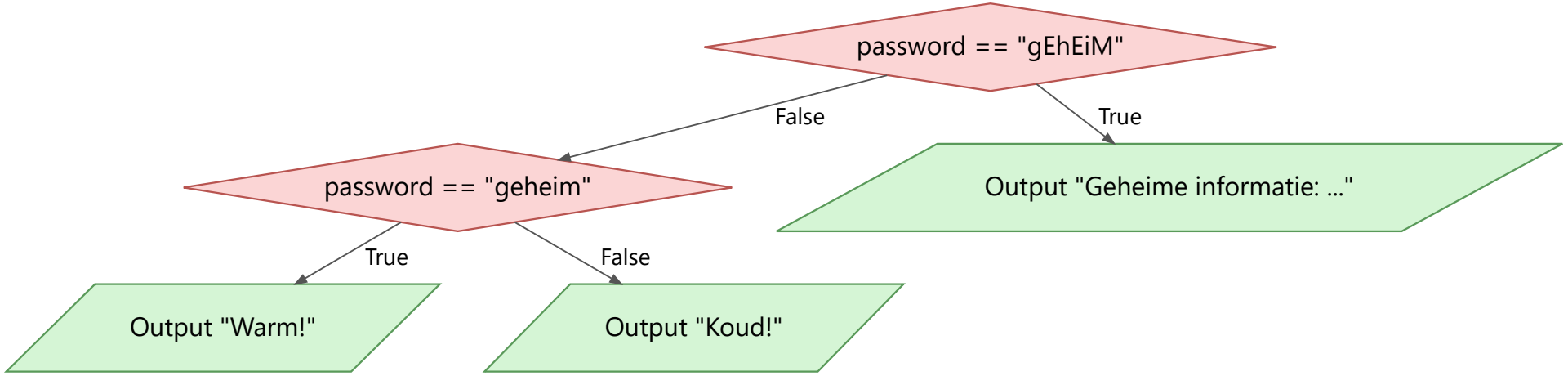
Nesting: een `if` in een `if`

```
if (password == "gEhEiM")
{
    Console.WriteLine("Geheime informatie: ...");
}
else
{
    Console.WriteLine("Dat klopt niet! Je krijgt nog één kans...");
    password = Console.ReadLine();
    if (password == "gEhEiM")
    {
        Console.WriteLine("Geheime informatie: ...");
    }
    else
    {
        Console.WriteLine("Ook niet juist.");
    }
}
```

Te diep nesten is als matroesjka's openen: leuk, tot je de kleinste zoekt.

Meerdere gevallen

Niet het juiste wachtwoord, maar **bijna**? Dan geven we een hint.



Genest in C#

```
1  if (password == "gEhEiM")
2  {
3      Console.WriteLine("Geheime informatie: ...");
4  }
5  else
6  {
7      if (password == "geheim")
8      {
9          Console.WriteLine("Warm!");
10     }
11     else
12     {
13         Console.WriteLine("Koud!");
14     }
15 }
```

Hetzelfde met `else if`

```
 1  if (password == "gEhEiM")
 2  {
 3      Console.WriteLine("Geheime informatie: ...");
 4  }
 5  else if (password == "geheim")
 6  {
 7      Console.WriteLine("Warm!");
 8  }
 9  else
10  {
11      Console.WriteLine("Koud!");
12  }
```

Doet **exact hetzelfde**, maar leest van boven naar onder als een lijstje gevallen.

Warm! Koud! Programmeren is eigenlijk "warmer, kouder" spelen met de compiler.

De volgorde telt ⚠️

```
1  int score = 17;
2  if (score >= 10)
3  {
4      Console.WriteLine("Geslaagd");
5  }
6  else if (score >= 16)
7  {
8      Console.WriteLine("Met onderscheiding!");
9  }
```

```
Geslaagd
```

`17 >= 10` is al `true`, dus de tweede voorwaarde wordt **nooit** bekeken. Zet de **strengste** voorwaarde eerst.

 *De eerste ware voorwaarde wint. Zoals aan de kassa: wie eerst komt, eerst maalt.*

Wat is de output? 🤔

```
1  if (temperature > 30)
2  {
3      Console.WriteLine("Tropisch!");
4  }
5  else if (temperature > 20)
6  {
7      Console.WriteLine("Lekker weer");
8  }
9  else if (temperature > 10)
10 {
11     Console.WriteLine("Jas mee");
12 }
13 else
14 {
15     Console.WriteLine("Blijf binnen");
16 }
```

`temperature`	Output
22	`Lekker weer`
35	`Tropisch!`
10	`Blijf binnen`

😈 *Bij 10 graden binnenblijven. Typisch Belgisch weerbericht.*

Handig voor de oefeningen: `%`

De **modulo**-operator `%` geeft de **rest** van een deling.

```
1  Console.WriteLine(10 % 3);  // 1
2  Console.WriteLine(14 % 5);  // 4
3  Console.WriteLine(8 % 2);   // 0 → 8 is even
4  Console.WriteLine(7 % 2);   // 1 → 7 is oneven
```

Een getal is **even** als `number % 2 == 0`.

😈 `%` *is geen korting. Sorry.*

Samengevat

Wat	Hoe
vergelijken	`<`, `>`, `<=`, `>=`, `==`, `!=` → `true` of `false`
soms iets doen	`if (voorwaarde) { ... }`
anders iets anders doen	`if { ... } else { ... }`
meerdere gevallen	`if { ... } else if { ... } else { ... }`

BELANGRIJK

Altijd **accolades**. En bij `else if`: de **eerste** ware voorwaarde wint, dus let op de volgorde.

Je kan nu beslissingen nemen. Begin maar met de moeilijkste: stoofvlees of bicky?

Aan de slag 💻

Maak in de klasse `Beslissingen`:

1. **ShoeSeller** (basis): bereken de prijs van schoenen, met korting vanaf 3 paar
2. **EvenOrOdd**: toon of een getal even of oneven is
3. **PositiveNegativeZero**: toon of een getal positief, negatief of 0 is
4. **BMICalculator**: bereken de BMI en geef een beoordeling

👉 Klaar? Dan **OhmsLaw** en **PocketMoney**. De uitbreiding van ShoeSeller volgt in labo 2.

😈 `if (exercisesDone) { naar huis } else { nog even blijven }`